

**UNIVERSIDAD PERUANA DE CIENCIAS
APLICADAS**



Facultad de Ingeniería
Carrera de Ciencias de la Computación

TEORÍA DE COMPILADORES (1ACC0218)

Sección: 4889

Docente: Jose Luis Soncco Alvarez

Kipu

*Un lenguaje de dominio específico para reglas tributarias y montos
monetarios*

Hito 1: Analizadores léxico, sintáctico y semántico (parcial) en ANTLR4

Integrantes

Bussalleu Salcedo, Fabrizio (U202315655)

Sancho Navarro Jose Luis (U202016523)

Marco Antonio Luciano Cabrera Díaz (U202318540)

<https://github.com/FabrizioBussalleu/kipu/>

Lima, octubre de 2026

1. Problemática y motivación

En el Perú, gran parte de las operaciones comerciales formales está sujeta a reglas tributarias como el IGV, las detracciones del SPOT, las retenciones y las percepciones. También existen operaciones que requieren trabajar con distintos tipos de cambio. En pequeñas empresas y sistemas de facturación electrónica, estas reglas suelen manejarse en hojas de cálculo o implementarse en lenguajes de propósito general. Las condiciones y porcentajes del sistema de detracciones dependen del tipo de operación y de los criterios definidos por SUNAT (SUNAT, s. f.).

Este enfoque puede generar varios problemas. Uno de ellos es el manejo de decimales y redondeos, ya que la aritmética de punto flotante puede producir resultados inesperados en algunos cálculos. Otro problema es que un programa común no impide, por sí solo, mezclar montos en soles y dólares. Además, las reglas de negocio pueden quedar repartidas dentro del código, lo que dificulta su lectura y revisión por personas que no programan. Finalmente, muchos errores se detectan recién cuando el programa se ejecuta.

Las herramientas existentes resuelven partes del problema, pero no necesariamente reúnen en un mismo lenguaje la validación de monedas, porcentajes y reglas tributarias. Por ese motivo, en este proyecto se propone Kipu, un lenguaje de dominio específico inspirado en el quipu. Kipu incorpora tipos como monto <PEN>, monto <USD> y porcentaje, de manera que el compilador pueda detectar antes de la ejecución operaciones incompatibles o asignaciones incorrectas.

2. Objetivos

Objetivo general. Diseñar e implementar un compilador para Kipu que traduzca reglas tributarias escritas en español a código ejecutable mediante ANTLR4 y LLVM, verificando durante la compilación la coherencia de tipos y monedas.

Para alcanzar este objetivo se plantean los siguientes objetivos específicos:

- ☐ **OE1.** Definir el léxico y las construcciones principales de Kipu, incorporando tipos de dominio como monto<moneda> y porcentaje, además de los tipos básicos.
- ☐ **OE2.** Especificar la gramática libre de contexto de cada construcción y comprobarla mediante derivaciones por la izquierda.
- ☐ **OE3.** Implementar los analizadores léxico y sintáctico en ANTLR4, incluyendo mensajes de error en español con línea y columna.
- ☐ **OE4.** Implementar un analizador semántico con tabla de símbolos por ámbitos y validación de tipos y monedas. En este hito se consideran cinco errores semánticos y se ampliarán en el Hito 2.
- ☐ **OE5.** Generar LLVM IR representando los montos como enteros de 64 bits en céntimos para evitar problemas de precisión en cálculos monetarios. Esta etapa corresponde al Hito 3.

3. Construcciones del lenguaje

Kipu utiliza tipado estático y explícito, bloques delimitados por llaves y sentencias terminadas en punto y coma. La precedencia de operadores, de mayor a menor, es: paréntesis, llamadas y convertir; menos unario; *, / y mod; + y -; operadores relacionales; igualdad; no; y; o.

Los montos pueden multiplicarse por números o porcentajes, y el cambio de moneda se realiza únicamente mediante convertir (monto, moneda, tipo_cambio). La Tabla 1 resume las siete construcciones principales y presenta cinco ejemplos para cada una.

Tabla 1. Construcciones de Kipu y ejemplos

Construcción	Ejemplos (5 por construcción)
G1. Declaración de variables y constantes var o const, nombre, tipo obligatorio y valor inicial (opcional en variables, obligatorio en constantes).	var cantidad: entero = 3; const IGV: porcentaje = 18%; var subtotal: monto<PEN> = 1250.50 PEN; var razonSocial: texto = "Elaris SAC"; var esAgenteRetencion: booleano;
G2. Asignación Asignación simple (=) y compuesta (+=, -=).	cantidad = 5; subtotal = precio * cantidad; total += igv; saldo -= 150.00 PEN; exonerado = falso;

Kipu | Hito 1 | 1

Construcción	Ejemplos (5 por construcción)
G3. Expresiones aritméticas, monetarias, relacionales y	precio * cantidad + flete base * 18%

lógicas Aritméticas, monetarias, relacionales, lógicas y conversión de moneda.	$(total - detraccion) / 2$ total $\geq$ 700.00 PEN y no exonerado convertir(neto, USD, 3.75)
G4. Sentencias selectivas si con cero o más sino si y un sino opcional; bloques obligatorios.	si (total > 700.00 PEN) { detraccion = total * 12%; } si (esExportacion) { igv = 0.00 PEN; } sino { igv = base * 18%; } } si (tipo == 1) { tasa = 12%; } sino si (tipo == 2) { tasa = 10%; } } sino { tasa = 4%; } si (no pagado y diasAtraso > 30) { mostrar("Factura vencida"); } si (saldo <= 0.00 PEN) { cancelado = verdadero; } sino { saldo -= cuota; }
G5. Sentencias iterativas mientras con condición y para con contador, límites y paso opcional.	mientras (saldo > 0.00 PEN) { saldo -= cuota; } para mes desde 1 hasta 12 { total += cuota; } para i desde 1 hasta n paso 2 { mostrar(i); } mientras (intentos < 3 y no validado) { intentos += 1; } para anio desde 2020 hasta 2026 { si (anio mod 4 == 0) { dias = 366; } }
G6. Reglas (funciones), retorno y llamadas Funciones tipadas con parámetros, retornar y llamada como expresión o sentencia.	regla calcularIgv(base: monto<PEN>): monto<PEN> { retornar base * 18%; } regla esMayor(a: entero, b: entero): booleano { retornar a > b; } regla imprimirCabecera() { mostrar("FACTURA ELECTRONICA"); } regla cuota(total: monto<PEN>, n: entero): monto<PEN> { retornar total / n; } regla neto(total: monto<PEN>, tasa: porcentaje): monto<PEN> { retornar total - calcularDescuento(total, tasa); }
G7. Entrada y salida mostrar con uno o más argumentos y leer sobre una variable.	mostrar("Total:", total); mostrar(subtotal + igv); leer(cantidad); mostrar("IGV", calcularIgv(base), "Neto", neto); leer(precioUnitario);

4. Analizador léxico

El léxico se define en la sección de lexer de grammar/Kipu.g4. Las palabras reservadas se declaran antes que ID para que ANTLR las reconozca correctamente cuando existe coincidencia. Cuando dos tokens comparten un prefijo, se aplica la coincidencia más larga.

De forma similar, LIT_PORCENTAJE aparece antes que NUM_DECIMAL y este antes que NUM_ENTERO. Los comentarios de línea y bloque, junto con los espacios en blanco, se descartan durante el análisis. Esta organización sigue el comportamiento habitual del lexer de ANTLR4 (Parr, 2013).

Tabla 2. Léxico de Kipu (token: lista de lexemas)

VAR: var	MOD: mod
CONST: const	SUMA: +
ENTERO: entero	RESTA: -
DECIMAL: decimal	MULT: *
PORCENTAJE: porcentaje	DIV: /
BOOLEANO: booleano	ASIG: =
TEXTO: texto	MAS_ASIG: +=
MONTO: monto	MENOS_ASIG: -=
MONEDA: PEN, USD, EUR	IGUAL: ==
SI: si	DIFERENTE: !=
SINO: sino	MENOR: <
MIENTRAS: mientras	MENOR_IGUAL: <=
PARA: para	MAYOR: >

DESDE: desde	MAYOR_IGUAL: >=
HASTA: hasta	PAR_IZQ: (
PASO: paso	PAR_DER:)
REGLA: regla	LLAVE_IZQ: {

Kipu | Hito 1 | 2

RETORNAR: retornar	LLAVE_DER: }
MOSTRAR: mostrar	PUNTO_COMA: ;
LEER: leer	COMA: ,
CONVERTIR: convertir	DOS_PUNTOS: :
Y: y	NUM_ENTERO: [0-9]+ (0, 3, 2026)
O: o	NUM_DECIMAL: [0-9]+ . [0-9]+ (3.75, 1250.50)
NO: no	LIT_PORCENTAJE: [0-9]+ (. [0-9]+)? % (18%, 12.5%)
VERDADERO: verdadero	CADENA: " ... " ("Elaris SAC")
FALSO: falso	ID: letra (letra dígito)*; letra incluye _ y áéíóúñ (subtotal, IGV, año)

Se ignoran los tokens **COMENTARIO_LINEA** (// ...), **COMENTARIO_BLOQUE** (/ * ... */) y **ESPACIO**. Un símbolo que no pertenezca al alfabeto permitido, como #, @ o \$, o una cadena que no se cierre correctamente produce el error LEX-01 con su línea y columna.

5. Analizador sintáctico

La Tabla 3 presenta la gramática libre de contexto usando la notación trabajada en el curso: no terminales, terminales, producciones y ϵ para la cadena vacía. La precedencia se separa por niveles, de modo que cada grupo de operadores tenga un orden claro.

En Kipu.g4 esta estructura se implementa con reglas compatibles con ANTLR4, incluyendo recursividad por la izquierda para expresiones y operadores EBNF para listas. Este enfoque permite que el parser construya un árbol sintáctico respetando la precedencia y la asociatividad de los operadores (Aho et al., 2007; Parr, 2013).

Tabla 3. Gramática libre de contexto de Kipu

G0. Estructura general del programa <i>programa</i> $\rightarrow$ <i>sentencias</i> <i>sentencias</i> $\rightarrow$ <i>sentencia sentencias</i> ϵ <i>sentencia</i> $\rightarrow$ <i>decl</i> <i>asig</i> <i>sel</i> <i>iter</i> <i>def_regla</i> <i>ret</i> <i>es</i> <i>llam_sent</i> <i>bloque</i> <i>bloque</i> $\rightarrow$ { <i>sentencias</i> } G1. Declaración de variables y constantes <i>decl</i> $\rightarrow$ var <i>id</i> : <i>tipo init</i> ; const <i>id</i> : <i>tipo = expr</i> ; <i>init</i> $\rightarrow$ = <i>expr</i> ϵ <i>tipo</i> $\rightarrow$ entero decimal porcentaje booleano texto monto < moneda > G2. Asignación <i>asig</i> $\rightarrow$ id <i>op_asig</i> <i>expr</i> ; <i>op_asig</i> $\rightarrow$ = += -= G4. Sentencias selectivas <i>sel</i> $\rightarrow$ si (<i>expr</i>) <i>bloque sino_parte</i> <i>sino_parte</i> $\rightarrow$ sino <i>sino_cola</i> ϵ <i>sino_cola</i> $\rightarrow$ <i>sel</i> <i>bloque</i> G5. Sentencias iterativas <i>iter</i> $\rightarrow$ mientras (<i>expr</i>) <i>bloque</i> para <i>id desde expr hasta expr paso_opc</i> <i>bloque</i> <i>paso_opc</i> $\rightarrow$ paso <i>expr</i> ϵ G7. Entrada y salida	G3. Expresiones <i>expr</i> $\rightarrow$ <i>expr o conj</i> <i>conj</i> <i>conj</i> $\rightarrow$ <i>conj y neg</i> <i>neg</i> <i>neg</i> $\rightarrow$ no <i>neg</i> <i>igu</i> <i>igu</i> $\rightarrow$ <i>igu op_igu rel</i> <i>rel</i> <i>op_igu</i> $\rightarrow$ == != <i>rel</i> $\rightarrow$ <i>rel op_rel arit</i> <i>arit</i> <i>op_rel</i> $\rightarrow$ < <= > >= <i>arit</i> $\rightarrow$ <i>arit op_sum term</i> <i>term</i> <i>op_sum</i> $\rightarrow$ + - <i>term</i> $\rightarrow$ <i>term op_mul unario</i> <i>unario</i> <i>op_mul</i> $\rightarrow$ * / mod <i>unario</i> $\rightarrow$ - <i>unario</i> <i>prim</i> <i>prim</i> $\rightarrow$ (<i>expr</i>) convertir (<i>expr</i> , moneda , <i>expr</i>) <i>llamada</i> <i>lit</i> id <i>lit</i> $\rightarrow$ <i>num moneda</i> num_ent num_dec lit_porc <i>cadena</i> verdadero falso <i>num</i> $\rightarrow$ num_ent num_dec <i>llamada</i> $\rightarrow$ id (<i>args</i>) <i>args</i> $\rightarrow$ <i>expr args_resto</i> ϵ <i>args_resto</i> $\rightarrow$, <i>expr args_resto</i> ϵ G6. Reglas (funciones), retorno y llamadas <i>def_regla</i> $\rightarrow$ regla id (<i>params</i>) <i>tipo_ret</i>
---	--

$es \rightarrow \text{mostrar} (expr \text{ args_resto }) ; \text{ leer} (id) ;$	$bloque$ $params \rightarrow param \text{ params_resto } \epsilon$ $params_resto \rightarrow , param \text{ params_resto } \epsilon$ $param \rightarrow id : tipo$ $tipo_ret \rightarrow : tipo \epsilon$ $ret \rightarrow \text{retornar} ret_expr ;$ $ret_expr \rightarrow expr \epsilon$ $llam_sent \rightarrow llamada ;$
---	---

Validación por derivaciones

Para cada construcción se derivaron por la izquierda cuatro de sus cinco ejemplos, obteniendo 28 derivaciones. Para reducir errores manuales, `scripts/derivaciones.py` tokeniza cada ejemplo con el lexer de ANTLR, lo reconoce con la gramática de la Tabla 3 mediante un parser Earley, verifica que el árbol sea único y luego lo recorre en preorden para generar la derivación lm.

El detalle se encuentra en el Anexo A (`docs/anexo_derivaciones.md`).

Ejemplo G2.1, cantidad = 5;;

$asig \Rightarrow id \text{ op_asig } expr ; \Rightarrow id = expr ; \Rightarrow id = conj ; \Rightarrow id = neg ; \Rightarrow id = igu ; \Rightarrow id = rel ; \Rightarrow id = arit ; \Rightarrow id = term ; \Rightarrow id = unario ; \Rightarrow id = prim ; \Rightarrow id = lit ; \Rightarrow id = num_ent ;$

6. Analizador semántico

El análisis semántico se implementa mediante un visitor sobre el árbol generado por ANTLR (`src/semantico.py`). El visitor mantiene una tabla de símbolos con ámbitos anidados para el nivel global, las reglas, los bloques y los ciclos para.

Las reglas globales se registran antes de analizar sus cuerpos, lo que permite realizar llamadas incluso cuando la definición aparece más adelante. Este tipo de verificación mediante tablas de símbolos y comprobación de tipos forma parte de las tareas habituales de un compilador (Aho et al., 2007).

Cada expresión produce un tipo. Entre las reglas principales se consideran:

$monto<M> \pm monto<M> \rightarrow monto<M>$

$monto \times (entero | decimal | porcentaje) \rightarrow monto$

$monto<M> / monto<M> \rightarrow decimal$

También se permite el ensanchamiento de entero a decimal.

Cuando una subexpresión contiene un error, su tipo se marca como desconocido para evitar que el mismo problema genere varios errores en cascada.

Tabla 4. Errores semánticos verificados en el Hito 1

Código	Error	Ejemplo	Mensaje del compilador (resumido)
SEM-01	Identificador no declarado (variable, constante o regla)	total = subtotal + igv;	'igv' no ha sido declarado.
SEM-02	Redeclaración en el mismo ámbito (incluye parámetros y contador del para)	var total: monto<PEN> = 10 PEN; var total: monto<PEN> = 20 PEN;	'total' ya fue declarado en este ámbito (línea 2).
SEM-03	Monedas incompatibles al operar, comparar o asignar montos	soles + dolares	no se puede sumar monto<PEN> con monto<USD>; use convertir(...).
SEM-04	Tipos incompatibles en asignación u operación	var n: monto<PEN> = "Elaris"; precio * precio	no se puede asignar texto a 'n' de tipo monto<PEN>.
SEM-05	Modificación de una constante (asignación o leer)	IGV = 10%;	No se puede modificar la constante 'IGV'.

Para el Hito 2 se plantea añadir nuevas comprobaciones: condición no booleana en si o mientras; retornar fuera de una regla o con un tipo diferente al declarado; cantidad o tipo incorrecto de argumentos en una llamada; regla con tipo de retorno que no retorna; porcentaje literal mayor a 100 %; y división entre cero cuando el divisor sea un literal.

7. Implementación y pruebas

El repositorio incluye grammar/Kipu.g4, el parser generado para Python en src/generated, el driver src/kipu.py, el analizador semántico, los ejemplos y las pruebas.

El driver ejecuta las tres fases en orden y no inicia el análisis semántico si antes se detectan errores léxicos o sintácticos. ANTLR4 se utiliza para generar el lexer y el parser a partir de la gramática definida para el lenguaje (Parr, 2013).

Para ejecutar el programa se utiliza:

```
python src/kipu.py ejemplos/facturacion.kp [--tokens] [--arbol] [--simbolos]
```

Las pruebas se ejecutan con el siguiente comando:

```
python -m unittest discover -s tests -v
```

Cada archivo .kp indica en su primera línea el resultado esperado, por ejemplo:

```
// espera: SEM-03@4
```

La Figura 1 resume un ejemplo en el que el programa contiene cuatro errores semánticos de distinto tipo. Cada error se informa con su código, línea y columna, y el compilador continúa el análisis para mostrar todos los problemas encontrados en una sola ejecución.

Figura 1. Programa con errores semánticos (izquierda) y salida del compilador (derecha)

<pre>1 // Consolidado anual de ventas 2 const UIT: monto<PEN> = 5350.00 PEN; 3 var ventas: monto<PEN> = 0.00 PEN; 4 var ventasUsd: monto<USD> = 0.00 USD; 5 para mes desde 1 hasta 12 { 6 UIT = 5500.00 PEN; 7 ventas += ventasUsd; 8 mostrar(totalAnual);</pre>	<pre>Kipu 0.1.0 (Hito 1) errores.kp [1/3] Análisis léxico OK (62 tokens) [2/3] Análisis sintáctico OK [3/3] Análisis semántico 4 error(es) [SEM-05] línea 6:5 No se puede modificar la constante 'UIT'. [SEM-03] línea 7:15 Monedas incompatibles: no se puede sumar monto<PEN> con monto<USD>; use convertir(...). [SEM-01] línea 8:13 'totalAnual' no ha sido declarado. [SEM-02] línea 9:9 'mes' ya fue</pre>
--	---

9 var mes: entero = 1; 10 }	declarado en este ámbito (línea 5). Resultado: 4 error(es) encontrados.
------------------------------------	--

Tabla 5. Casos de prueba del Hito 1 (todos aprobados)

Analizador	Casos	Resultado
Léxico	6 archivos (palabras clave, literales, operadores, identificadores y comentarios, símbolos inválidos, cadena sin cerrar) y 5 secuencias de tokens	11 / 11 correctos
Sintáctico	35 ejemplos de la Tabla 1, 3 programas válidos y 9 inválidos (falta ';', paréntesis sin cerrar, tipo inexistente, sino sin si, etc.)	47 / 47 correctos

Kipu | Hito 1 | 4

Analizador	Casos	Resultado
Semántico	4 programas válidos (ámbitos, operaciones monetarias, reglas, facturación) y 6 inválidos con 19 errores esperados	10 / 10 correctos

8. Referencias

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2007). *Compilers: Principles, techniques, and tools* (2.^a ed.). Pearson.

Parr, T. (2013). *The definitive ANTLR 4 reference*. The Pragmatic Bookshelf.

Superintendencia Nacional de Aduanas y de Administración Tributaria [SUNAT]. (s. f.). *Detracciones del IGV: Sistema de pago de obligaciones tributarias* [Cartilla]. https://orientacion.sunat.gob.pe/sites/default/files/inline-files/Cartilla_detracciones.pdf